



**LEVERAGING SENTIMENT ANALYSIS FOR CONFLICT MONITORING IN SUB-
SAHARAN AFRICA**

APPLIED PROJECT

B.Sc. Computer Science

Yasmin Angaa-Mwini Kamal-Deen Gbontaa

2024

ASHESI UNIVERSITY

**LEVERAGING SENTIMENT ANALYSIS FOR CONFLICT MONITORING IN SUB-
SAHARAN AFRICA**

UNDERGRADUATE APPLIED PROJECT

**Undergraduate thesis submitted to the Department of Computer Science & Information
Systems, Ashesi University in partial fulfilment of the requirements for the award of
Bachelor of Science degree in Computer Science.**

Yasmin Angaa-Mwini Kamal-Deen Gbontaa

2024

Declaration

I hereby declare that this Applied Project is the result of my original work and that no part of it has been presented for another degree in the university or elsewhere.

Candidate's Signature:

.....

Candidate's Name:

.....

Date:

.....

I hereby declare that the preparation and presentation of this Applied Project were supervised in accordance with the guidelines on the supervision of Applied Projects laid down by Ashesi University.

Supervisor's Signature:

.....

Supervisor's Name:

.....

Date:

.....

Acknowledgement

I am grateful to Allah and to family, friends, and colleagues that have encouraged and supported me during this period.

I would like to thank my supervisor, Dr. Tatenda Kavu, for his guidance and insights throughout the project. I would also like to thank Kwaku Andoh Yamoah and Akwasi Krobea for their constant availability and support.

ABSTRACT

Conflict monitoring systems provide a means to collect data and analyze conflict. With 570 million internet users in the sub-Saharan Africa region as of 2022, there is an opportunity to utilize social media data for conflict monitoring in the area. Recognizing the need for a conflict monitoring system that enables the monitoring of conflict to support quick response and action, the project's objective is to build a system that uses sentiment analysis to analyze the occurrence of protests and events in a given geographical area. The proposed system uses machine learning approaches to extract sentiments on social media and predict the occurrence of protests and different event types. The project utilizes the Pysentimiento library based on BERT transformer models for sentiment analysis. The two Multilayer Perceptron models utilized BERT embeddings to predict protests and event types, the weighted F1 score was 0.7219 and 0.6440, respectively. Given the use case of Uganda, the models correctly predicted the protests and other event types. The project also highlights the potential to use machine-learning sentiment analysis methods for real-life conflict monitoring in sub-Saharan Africa.

Table of Contents

Acknowledgement.....	ii
List of Figures	vi
List of Tables	vii
List of Appendices	viii
CHAPTER 1: INTRODUCTION.....	1
CHAPTER 2: REQUIREMENT ANALYSIS.....	7
CHAPTER 3: DESIGN AND ARCHITECTURE	16
CHAPTER 4: IMPLEMENTATION.....	21
CHAPTER 5: TESTING AND RESULTS	42
CHAPTER 6: CONCLUSIONS AND RECOMMENDATIONS.....	47
References	50
Appendix.....	52

List of Figures

Figure 2.1. Use Case diagram for system end users	11
Figure 2.2. Use case diagram for machine learning engineer.....	12
Figure 2.3. Use case diagram for DevOps Engineer.....	13
Figure 3.2. Component Architecture Diagram	18
Figure 3.3. Module Architecture Diagram.....	20
Figure 4.2. Data Collection Process for Model Training.....	22
Figure 4.3. Distribution of data count across countries	23
Figure 4.4. Distribution of disorder type in training dataset.....	24
Figure 4. 5. Distribution of disorder type in training dataset.....	25
Figure 4.6. Adding features to tweets for prediction	26
Figure 4.7. Creating numerical representation for embedding	27
Figure 4.8. Model for changing features to embedding.....	28
Figure 4.9. Embedding for texts	28
Figure 4.10. Combining embedding for training	28
Figure 4.11. Forward pass for combined embedding	29
Figure 4.12. Training loop and other methods for model	30
Figure 4.13. Defining parameters for GridSearchCV	31
Figure 4.14. Training with GridSearchCV	32
Figure 4.15. Model definitions for disorder type and event type.....	32
Figure 4.16. K-fold Cross Validation Loop	33
Figure 4.17. Backend Application process	34
Figure 4.18. Filtering tweets by unrest score	35
Figure 4.19. Extracting hashtags.....	36
Figure 4.20. Predicting using model	36
Figure 4.21. Aggregating data using majority vote	37
Figure 4.22. Initializing socket for communication.....	38
Figure 4.23. Using socket for communication.....	38
Figure 4.24. Select country page	39
Figure 4. 25. View dashboard page	39
Figure 4.26. User download of report file	40
Figure 4.27. Socket communication on frontend application.....	40
Figure 4. 28. Updating predictions, confidence scores, and keywords when data is received from socket	40
Figure 4. 29. Update arc when data is updated	41
Figure 4.30. Update word cloud when keyword is updated	41
Figure 5.1. Frontend message to server	42
Figure 5.2. Backend application response to frontend application	43
Figure 5.3. The frontend component connected to the backend server	43
Figure 5.4. The backend component connected to the frontend component	43
Figure 5.5. Backend endpoint component testing.....	44
Figure 5. 6. System Testing steps	45
Figure 5.7. Results for Kenya from ACLED API.....	45

List of Tables

Table 4.1. Evaluation results for disorder-type model.....	33
Table 4.2. Evaluation results for event-type model	33

List of Appendices

Appendix A: Semi-Structured Interview Guide	52
Appendix B: Register and Login Implementation	54
Appendix C: Register and Login for User	55
Appendix D: Select Country	55
Appendix E: Real time monitoring of Kenya Tweets.....	56
Appendix F: Real time monitoring of Uganda Tweets	56
Appendix G: Downloading report file	57
Appendix H: Login component testing.....	57
Appendix I: Register component testing	57

CHAPTER 1: INTRODUCTION

1.1. Background

Sub-Saharan Africa has been plagued with more armed conflicts in the world between 2020 and 2022 than other regions [16] in the world. The many negative effects of conflict include stunted economic growth, increased mortality rates, and mass displacement of citizens [8].

1.1.1. Conflict Monitoring Systems

Due to the negative effects of conflict on the development and stability of different nations, various research has been conducted on how monitoring systems can play a vital role in preventing, managing, and analyzing conflict. Conflict monitoring can be described as a process that detects and analyzes conflict to support decision-making. Traditional forms of conflict monitoring in the sub-Saharan region of Africa include experts, news outlets, and information from governmental bodies. These monitoring systems may be inconsistent or biased towards different actors. A study on conflict predictions from 300 experts in politics and economics revealed that the performance of these predictions was average and no better than random guesses [17]. Another study showed that when predicting occurrences in conflicts, while experts performed slightly better than novices, neither group's predictions were better than guesses [9].

The role of technological systems in monitoring was an evolving theme in recent research on preventive measures for disasters. While the impact of using technology to monitor conflict depends on the context of the conflict and availability of data, the common consensus is that the ability to manipulate data, visualize dynamics, and recognize patterns and signatures for conflict creates new opportunities for stakeholders like humanitarian and governmental

organizations [12]. Different organizations have systems that perform conflict monitoring using either technology or a mix of technological and traditional methods. In the context of Africa, the African Union (AU), in consultation with the United Nations, established the fifteen-member Peace and Security Council to serve as an early warning system in the region [3]. Following this decision, the AU created the AU Continental Early Warning System (CEWS), which relies on analytical and news sources and online news sites and articles. While this system is functional, its effectiveness continues to be hindered by limited staff capacity and expertise, material, and technical equipment [4]. Apart from the CEWS, programs like the Uppsala Conflict Data Program (UCDP) and Armed Conflict Location & Event Data Project (ACLED) support conflict monitoring in sub-Saharan Africa by providing publicly available data on ongoing conflict in the region.

The most common conflict monitoring system classification is the stage of conflict the system addresses. The main stages of conflict are before the conflict, during the conflict, and after the conflict. There are monitoring systems that focus on the period before the conflict to address mitigation and preparedness, those that focus on the conflict while it is ongoing to address response, and those that focus on the period after the conflict focuses on recovery [4]. These systems serve different purposes however they may be integrated into one system. This paper presents a conflict monitoring system that analyzes data during conflict to improve conflict response.

1.1.2. Social Media and Conflict Monitoring

The internet's popularity has spread worldwide, with more people joining the large network of devices every year. The amount of information shared and stored through the network has led to the internet being a gold mine for data, especially with the recent popularity of big data

for solutions. Out of the internet users around the globe, Africa makes up 13%, which amounts to around 570 million internet users as of 2022 [1]. As demonstrated by the African Union CEWS, the internet is an excellent source for data collection due to the speed of information sharing. In politics and conflict, social media has encouraged independent news sharing and mobilization of social groups towards a cause. Social media has played a significant role in popular political events such as the Arab Spring. During the Arab Spring, social media was used to recruit civilians into groups that eventually overthrew the democratic governance of the region [7]. Intelligence organizations did not anticipate this event, and this led to an unprepared response. From this example, it can be recognized that social media can harbor information that may be hard to collect from other sources. This characteristic of social media data is mainly because of users' proximity to events and the ability to capture events from different perspectives.

Social media data is usually in the form of text and can be analyzed by sentiment analysis. Sentiment analysis is an essential tool in conflict monitoring because it processes text to decipher emotional tones and opinions expressed within written texts presenting sentiments such as positivity, negativity, or neutrality. In the context of conflict monitoring, this method can be used to scrutinize media posts to unveil valuable insights into the response of a population to ongoing conflicts.

The project's main objective is to predict conflict in real time using machine learning and social media data.

1.2.Related Work

The use of social media data and sentiment analysis as a tool for prediction and analysis has been used in diverse topics of interest including political sentiments and disaster monitoring.

An example of this application is the use of posts from Sina Weibo, a popular Chinese social networking site, to analyze the occurrence of the 2014 Shanghai New Year's Eve stampede [6]. The proposed method by the authors used sentiment analysis along with topic modeling and special autocorrelation to categorize posts and predict when the stampede occurred in near real-time. The results of this study propose that sentiment analysis is a useful method to perform real-time conflict monitoring to determine the occurrence of events. The system proposed will use similar methods to perform conflict monitoring in Africa. While this study shows an application to this project, it differs slightly in the methods being used. The approach used in this project considers only sentiment analysis and uses conflicts in Africa as a case study.

Another system employs sentiment analysis to predict human action without considering the location of the social media post [11]. This study uses lexicon-driven methods of sentiment analysis and resulted in being able to predict the probability of four distinct types of action including the possible location of the event and the exact time the predicted event is going to happen. This project covers a good scope and shows an application of sentiment analysis for conflict prevention in Africa. However, the tool does not consider the real-time analysis of conflict in Africa and the possible implications of the predicted events to the community. The system proposed aims to fill this gap by considering social media posts that were generated during conflicts. The proposed system also fills a method gap by using a machine learning-based method for predicting the occurrence of conflict in real time.

1.3.Conflict Prediction System

Despite the growing prevalence of armed conflicts in sub-Saharan Africa, traditional conflict monitoring systems need to be revised in their effectiveness. The reliance on sources such as news and governmental bodies may need to be more consistent and limited due to biases or inaccessibility to conflict zones. Due to these problems, conflict monitoring systems that could support peace in sub-Saharan Africa could be more effective. The ineffectiveness of these systems means that potential actions to avoid, minimize, or recover after conflicts may be slow. Researchers have used technologies such as remote sensing and geographical information systems to address this issue; however, using sentiment analysis to explore conflict in sub-Saharan Africa remains an under-explored area.

The emergence of the internet and the rapid expansion of social media presents a valuable opportunity for more dynamic and real-time data collection in sub-Saharan Africa. Social media is a vast and interconnected platform that can provide unique insights into the complex nature of conflict. This applied project is intended to show that sentiment analysis using social media can be a valuable tool for conflict monitoring.

The objective of this project is to attempt to build a system that uses only sentiment analysis as a feature to predict the occurrence of different disorders and event types as defined by ACLED in their codebook [2]. This system will rely on machine learning models to extract sentiments and define the categories of the disorders and events.

The system proposed in this paper can be useful to national and multinational organizations worldwide. The research can be used to observe conflict dynamics and support strategies for subduing conflict, which contributes to peacebuilding. The tool developed can be used to

enhance humanitarian support to civilians affected by the ongoing disaster. Researchers and academics can use this research as a framework to build similar tools.

CHAPTER 2: REQUIREMENT ANALYSIS

This chapter provides insight into the process of deciding the appropriate features of the conflict monitoring system. The chapter considers how the user and the system will interact to ensure that the user can perform conflict monitoring using the system.

2.1. Requirements Elicitation

The requirements elicitation process for building this system includes reviewing already existing systems for requirements and communicating with the target users of the system to understand their current challenges and needs. The data from the requirements elicitation process is used to create user stories and use cases to properly define how the system works to address the users' needs.

2.1.1. Stakeholder Requirements Gathering

The main stakeholders of the conflict monitoring are humanitarian organizations, international organizations, the government, law enforcement agencies, and the people and businesses affected by the conflict. These stakeholders can impact conflict in separate ways. Stakeholders like humanitarian organizations, international organizations, the government, and law enforcement agencies can directly influence the dynamics of the conflict through the decisions they make. Investors like businesses and the people are directly affected by the conflict and often suffer the direct consequences of the conflict more than other stakeholders. This project acknowledges that due to the variety of stakeholders, there may be varying interests or opinions. The interaction with the stakeholders seeks to merely understand what data or information is important to each stakeholder and how this can be integrated into one system that can suit all stakeholders.

2.1.2. Semi-structured Interviews for Requirements Gathering

Semi-structured interviews were used as the tool to collect information from the stakeholders. This is the preferred tool because it allows the stakeholders to provide longer, more detailed responses that may yield relevance to the project. They also allow the interviewer to assess the relevance of the response to their project. The interviews were guided by a set of questions surrounding the need for a conflict monitoring system and the possible features of such a system. These can be found in Appendix A. Interviewees were selected based on legitimacy and their urgency towards the elicitation process. In total three one-on-one interviews with conflict experts were conducted, one in academia, and two in international organizations.

Main insights from interviews:

- Conflict monitoring in Africa is still mainly traditional
- While conflict monitoring systems supported by technology exist, these systems are not used effectively to monitor conflicts
- People in the environment of the conflict and affected by the conflict usually have more useful information about the conflict
- The intensity of a conflict has no definition and depends on what the international community deems to be intense. This means that conflicts within the Sub-Saharan Africa region that may have factors of intensity such as loss of lives may not be regarded as intense if they have no effects on the international community

- An easy-to-use system that attempts to monitor conflict may be useful and a step towards the widespread use of technology for easy conflict monitoring in Sub-Saharan Africa

These insights justify the need for a conflict monitoring system that uses sentiment analysis in Sub-Saharan Africa.

2.2.Users and Use Cases

The system's primary users are national and international organizations concerned with security, the safety of citizens, and the maintenance of order. In Ghana, such organizations include the police and the armed forces.

2.2.1. Use Case Description

Actors: Security Officer

Officer A is a security officer in Ghana. Recently, their organization acquired a conflict monitoring tool that allows them to analyze the possibility of unrest occurring in an area based on the social media data collected from this area. They use this tool with other tools to prepare for defense to ensure order and protect citizens.

Trigger

The security officer accesses the conflict monitoring tool on the web and logs into the platform.

Preconditions

1. The security officer must have access to the internet
2. The security officer must have a device capable of accessing the web

Main Success Path

The security officer selects the geographical area they want to observe. After some time, the system provides keywords observed in users' tweets and the probability of various disorders and events happening. These are accompanied by a confidence score of the predictions. The security officer can make decisions based on these predictions. The security officer can choose to download the report for analysis later.

Post Conditions

Success Guarantee – The predictions of the system are correct, and the organization is well prepared to manage the current conditions and outcomes of the conflict.

Minimal Guarantee – The system should be accessible at any time

Exception Path

The system is inaccessible to the user

The system does not provide any predictions given a selected area of interest

2.2.2. Use Case diagram for End Users (stakeholders) of the system

This use case diagram shows the actions that can be taken by the end users.

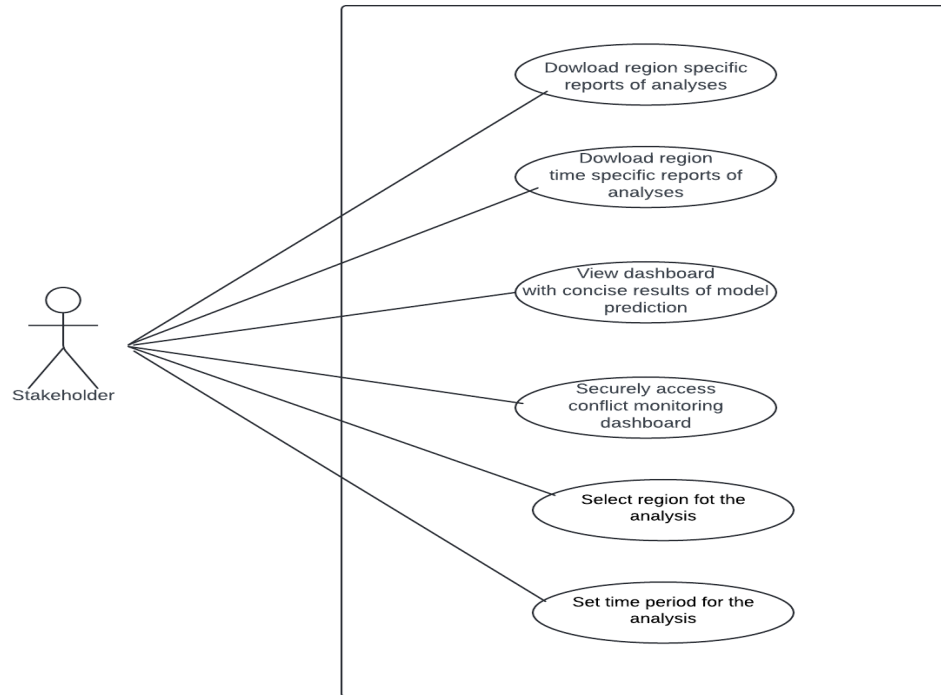


Figure 2.1. Use Case diagram for system end users

2.2.3. Use Case for Machine Learning Developer

This diagram shows the actions the machine learning engineer must take to ensure the system works.

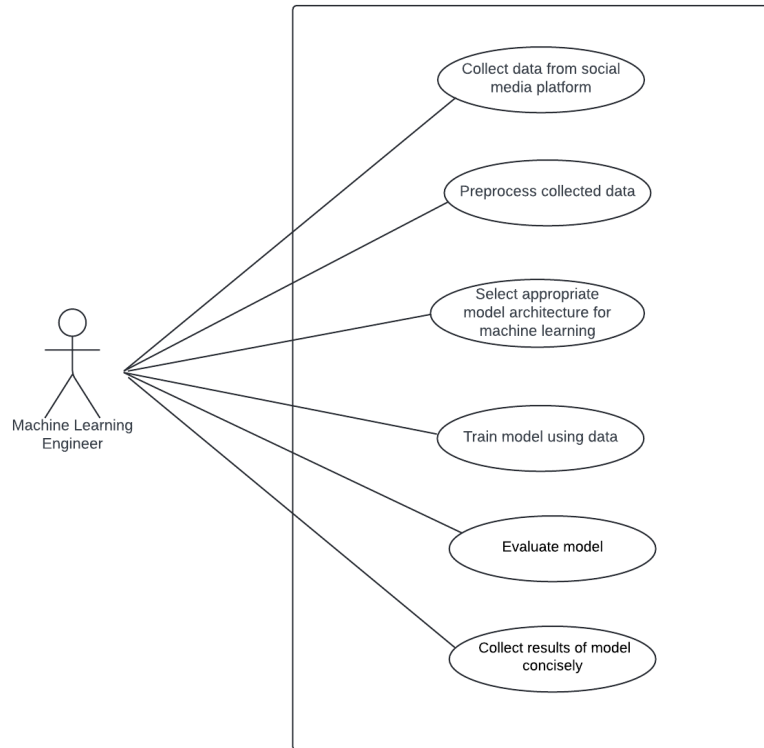


Figure 2.2. Use case diagram for machine learning engineer

2.2.4. Use case for DevOps engineer

This diagram shows the actions the DevOps engineer must take to ensure the system is available to the end user.

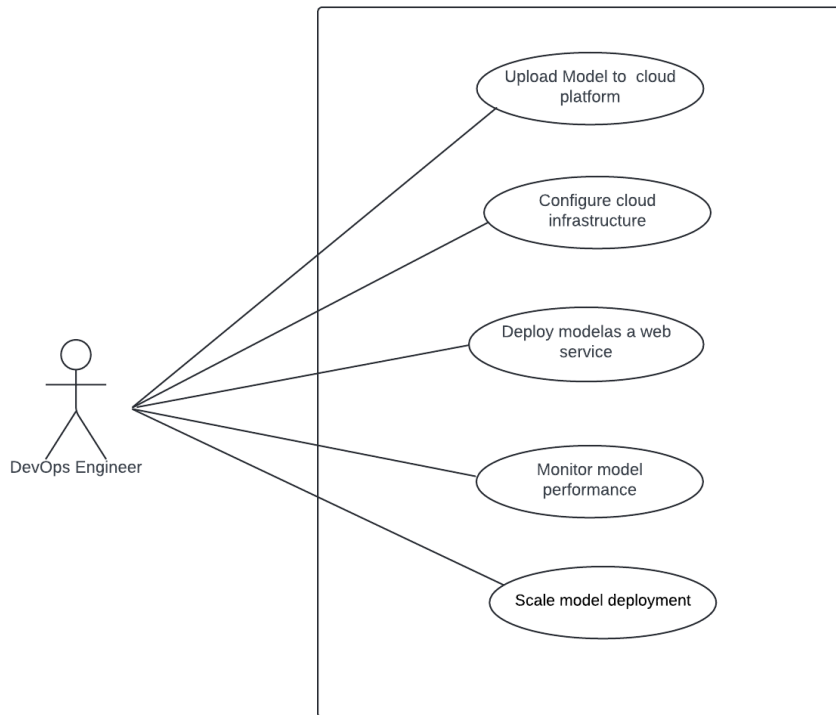


Figure 2.3. Use case diagram for DevOps Engineer

2.3. Functional Requirements

The following functional requirements are the basis for the system's development and proceed from the various use cases developed above. The functional requirements are as follows:

FR001: The system shall provide a web-based user interface for viewing results of model inference

FR002: Users shall be able to specify geographical regions of interest with Sub-Saharan Africa to generate crisis-related reports

FR003: Users shall be able to view reports containing information on the probability of various unrest happening

FR004: Users shall be able to download region specific reports of conflict

FR005: Users shall be able to download time specific reports of conflict

FR006: The system shall provide functionality for collecting and preprocessing data for model training

FR007: The system shall enable training and evaluating machine learning models using the collected and preprocessed data, providing metrics for model performance evaluation

FR008: The system shall support deploying the machine learning model as a service on a cloud platform

FR009: The system shall allow DevOps engineers to scale model deployments based on usage and performance metrics

2.4. Non-functional Requirements

NFR001: The system shall be user-friendly, with an intuitive interface for easy interaction and navigation

NFR002: The system shall respond promptly to user inputs, providing timely feedback and results

NFR003: The system shall ensure low latency for model inference requests to provide a smooth user experience

NFR004: The system shall be compatible with various web browsers and devices

NFR005: The system shall be scalable to handle varying user loads and data processing requirements during crisis events

NFR006: The system shall implement security measures to prevent unauthorized access and attacks to the dashboard and its functionalities, models, data, and infrastructure

NFR007: The system shall ensure high availability and reliability of deployed models to minimize downtime

2.5.Constraints

CO01: The system must operate on all mobile devices and all browsers.

CO02: The system must be accessible if the user has access to the Web.

CO03: The system development costs should be within the organization's budget.

CHAPTER 3: DESIGN AND ARCHITECTURE

Given the requirements of this project, the system will primarily be a two-tier architecture. The two-tier architecture has the client side and the server side that consists of the logic and the database [10]. This architecture supports real-time data processing and fast delivery of data to the client side. This architecture is also useful since the application uses just two use cases to show its functionality.

3.1. Components of System

The sentiment analysis system being created will comprise the typical machine learning components that make for a system meant for the user.

3.1.1. Data Collection Component

This system will provide data from both offline and online sources. The offline sources are databases that will store the data for preprocessing and training. The online source is the Twitter API, which will provide live tweets from a specified geographical area to predict conflict in a chosen geographical area and report live results to the system user. This service will support both structured and unstructured data to improve the system's robustness.

3.1.2. Data Cleaning Component

The second component of this system performs data cleaning. Data collected for training, testing, and user usage must be cleaned appropriately for the model to be trained for the task at hand. Apart from cleaning the model, we perform feature engineering to extract information that can be processed through training.

3.1.3. Model Training Component

Data that has undergone preprocessing and feature engineering can then be used for training using the sentiment analysis algorithm. In this component, data is trained to produce output that would be of most use to the users of the system. Apart from training the data, iterative testing is performed to improve the hyper-parameters and improve the overall performance of the model to make it more robust to the user.

3.1.4. User Services Component

The results of the sentiment analysis model are used by the report service and the dashboard service to make the system more user-friendly for people with varying experience using machine learning models for analysis. For this to happen, the model needs to be deployed to the cloud with its performance constantly monitored by the machine learning engineer.

These different services or components make up the server side and the client side. The diagram below represents the components in the two-tiered architecture by showing how the components interact with each other to form a complete system.

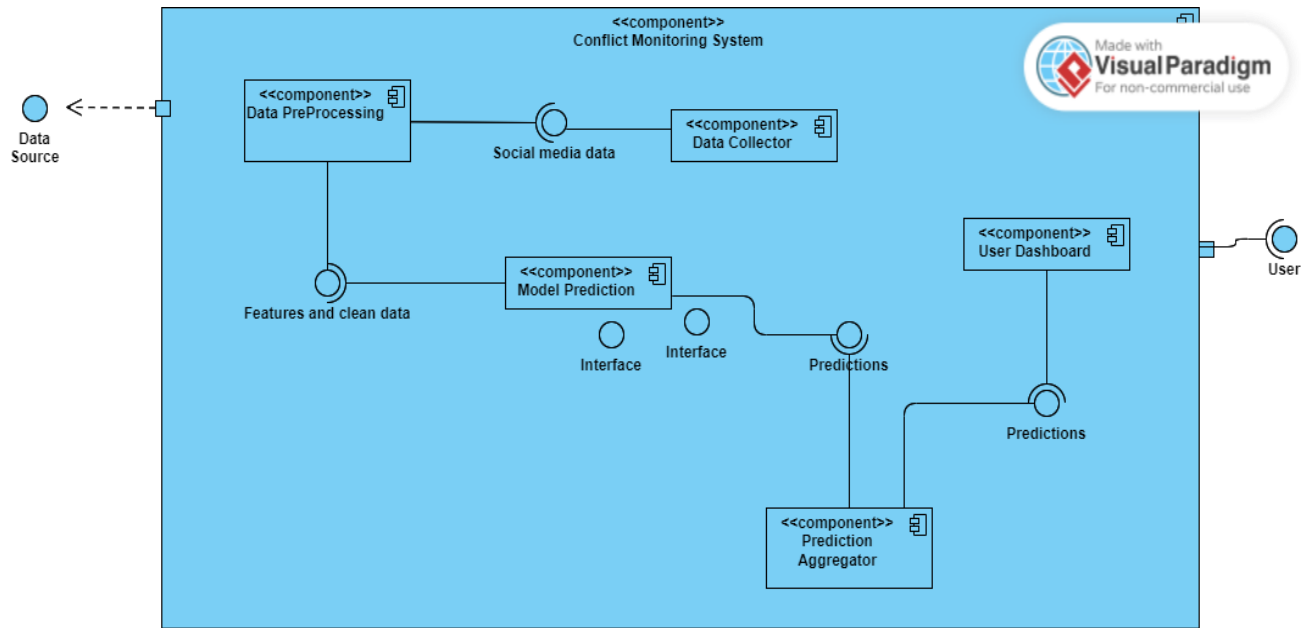


Figure 3.1. Component Architecture Diagram

3.2. Modules of Components in the System

The system's various components are made up of sub-components or modules that perform multiple tasks in them. This modularity allows for easy management of the system.

3.2.1. Data Collection Modules

The data collection component includes a Twitter client that handles communication with the Twitter API. The Twitter client sends tweets to the Tweet Processor sub-module to process raw tweets and put them in storage. A scheduler module oversees scheduling tweet collection tasks.

3.2.2. Cleaning and Feature Extraction Modules

The tweets from the data collection modules are filtered according to whether they relate to unrest or not. Feature extraction is performed on the tweets to include the sentiment

features that are useful to the prediction model. The keyword module extracts relevant keywords from the tweets.

3.2.3. Sentiment Analysis Modules

The preprocessing module transforms the text into inputs useful for the model. The model loader loads the pre-trained machine learning model, and the analyzer performs analysis on the pre-processed data. The analyzer interacts with the metrics calculator to estimate the model's performance. The analyzed data gets stored.

3.2.4. Report Generation Modules

The data aggregator and report builder modules work together to put the data into a format that can be exported and stored.

3.2.5. Web Dashboard Modules

The API controller handles API requests from the front end to the other components and services of the system. The authentication system ensures that authorized users are accessing the system. The API controller communicates with the dashboard renderer and data fetcher to display information to the user. The storage module manages sessions and user configurations.

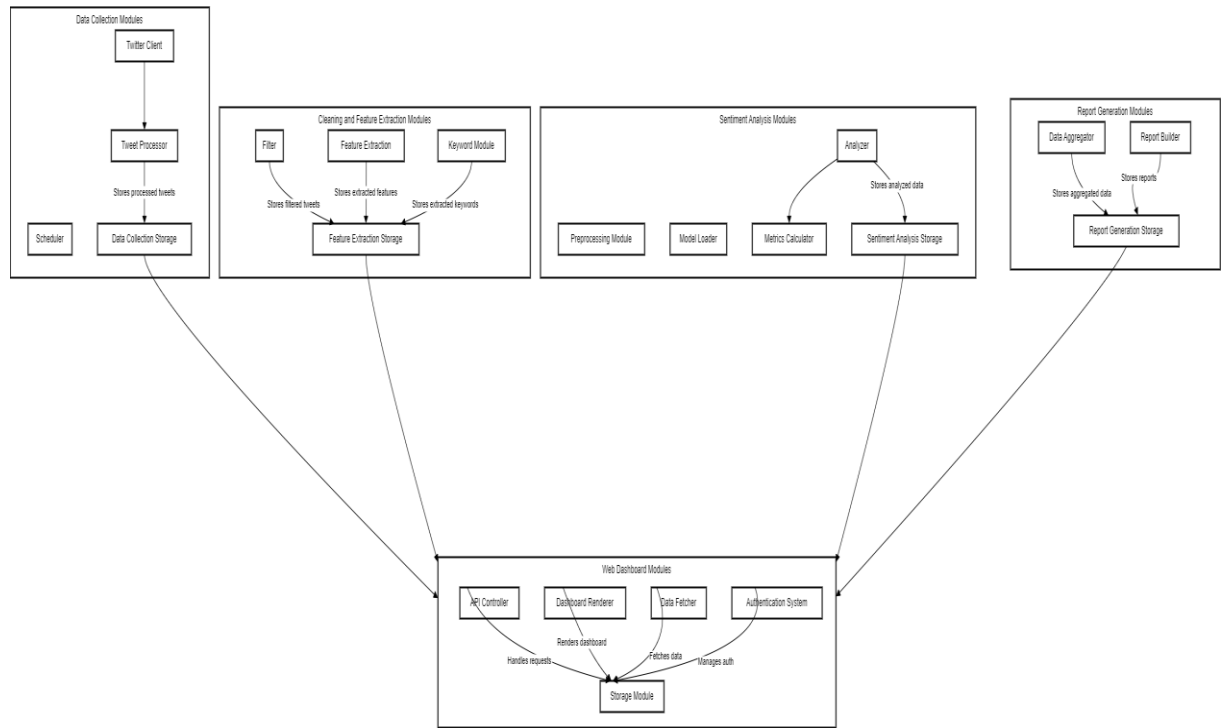


Figure 3.2. Module Architecture Diagram

CHAPTER 4: IMPLEMENTATION

The implementation of the prediction system consists of two main parts. The first results in a trained model for conflict-type prediction and the second results in a dashboard that allows users to interact with the model and view predictions in real-time. To build a model that could perform these predictions, the general problem of determining disorder type and event type had to be rephrased as a classification problem where each tweet is classified as belonging to a civil unrest event group. Following this classification, the tweets are aggregated based on probabilistic methods to inform the user of the probability of conflicts happening in a location.

4.1. Training Disorder Type and Event Type Classification Models

4.1.1. Data Collection

The dataset used for the training and validating this model is the Global Civil Unrest on Twitter (G-CUT) dataset [5]. The dataset contains over 3 million tweets from Africa, the Middle East, and some parts of Asia, from 2014 to 2019, making it suitable for use in a sub-Saharan Africa context. Each entry in the dataset is characterized by a tweet identification number, the country code, the time it was created, and a civil unrest score assigned to each tweet. The civil unrest score is a probability between 0 and 1 that a tweet talks about a civil unrest event happening. These scores are based on a transformer-based model fine-tuned on BERTweet with F1 score of 0.814 respectively for their test dataset [15].

While all 3 million could have been used to train a potentially powerful model, the volume of tweets that could be used is heavily restricted by the API access provided by the Twitter API. The Twitter API is a tool that allows subscribers to perform operations on Twitter objects

through API calls. Due to the restricted number of API calls allowed, the goal was to collect about 4000 tweets for testing and validation combined.

The first step towards collecting the tweets involved filtering the original dataset entries by civil unrest score provided by a third-party transformer model [15]. All tweets with civil unrest scores below 0.5 were discarded with the assumption that those tweets did not discuss civil unrest. Since the tool is specifically designed for sub-Saharan Africa, the goal was to train the model on tweets from countries in this region. Tweets from seven different English-speaking countries in the Sub-Saharan Africa region were chosen based on availability. Out of these tweets, focused sampling was performed to ensure that the most relevant tweets would have to be selected. To practice focused sampling, tweets were filtered based on dates when a civil unrest activity happened in the represented countries. This data were collected from Wikipedia and various online news sources. Among these entries, 830 entries were randomly sampled from each event to provide a uniform representation across different events.

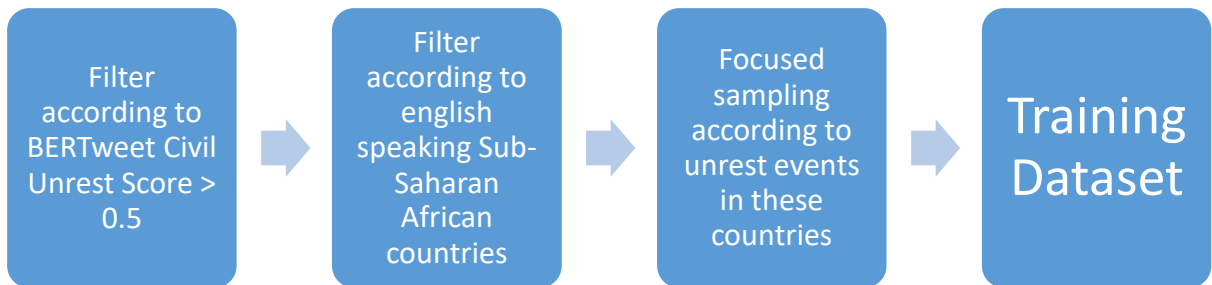


Figure 4.1. Data Collection Process for Model Training

From the resulting dataset, a new dataset that includes the text of the tweets was collected. API calls to the Twitter API were made using Tweepy, an open-source library that allows users to interact with the Twitter API using Python. The total size of the resulting dataset is 4919 unique entries. The countries included in the dataset are South Africa, Cameroon, Sierra Leone, Ghana, Nigeria, Kenya, and Uganda.

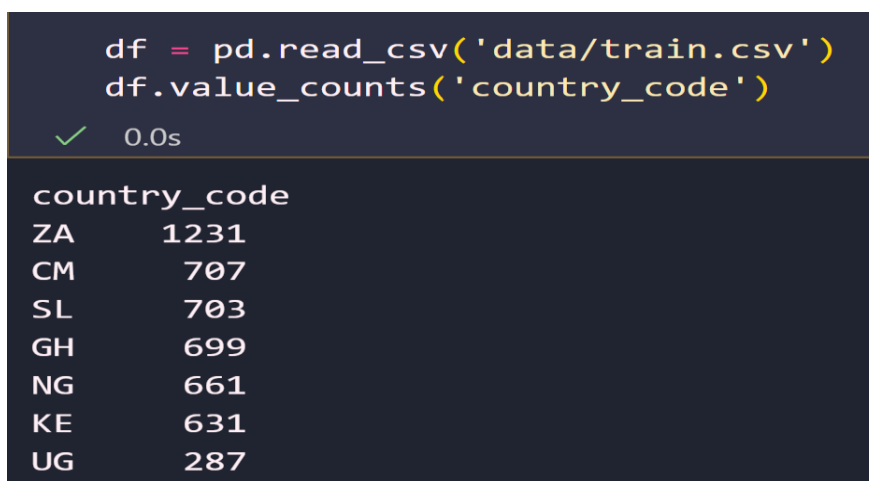


Figure 4.2. Distribution of data count across countries

4.1.2. Labeling Data

A major challenge was correctly labeling the tweets for training. While the data was labeled with civil unrest scores, it was not labeled with the civil unrest activity that could have been occurring at the time. Data from the Armed Conflict Location and Event Data (ACLED) organization was used to label individual tweets. ACLED is an organization that collects data on violent conflict and protests in all countries. The ACLED API was used to collect data on a disorder and event from their database. According to ACLED, the disorder type represents the type of disorder an event falls into and has 3 classes – Demonstrations, Political Violence, or

Strategic Developments [2]. There are 24 event types, however due to the limitation of our small dataset only 6 were represented. The 6 event types represented are Protests, Violence against Civilians, Explosions/Remote Violence, Battles, Strategic Developments, and Riots.

To label a tweet, a request is sent to the ACLED API with the country and date of the tweet that requires tagging. The tweet is then tagged with the disorder type, event type, and sub-event type that is returned by the ACLED API for that time. When multiple events are returned by the API based on the information sent, the event that happens closest to the time when the tweet was created is picked as the most relevant event and is used to tag the tweets. In the case that no event was recorded by ACLED, the tweet is tagged with No Relevant Event. This tag is important because it indicates situations where social media data may be related to unrest, however no unrest was happening at that time.

```
df.value_counts('disorder_type')  
✓ 0.0s  
  
disorder_type  
Demonstrations      2287  
Political violence   1764  
No relevant event    807  
Strategic developments    61  
Name: count, dtype: int64
```

Figure 4.3. Distribution of disorder type in training dataset



Figure 4. 4. Distribution of disorder type in training dataset

4.1.3. Feature Extraction

Since the goal of the system is to use sentiment analysis to predict the occurrence of conflict, sentiment features had to be extracted from each tweet. Apart from positive, negative, and neutral sentiment analysis, emotion and hate speech features are extracted for each tweet. The models used for extracting the features are based on the BERT transformer models [13]. The sentiment analysis and hate speech models are based on BERTweet, and the hate-speech model is based on the BERT base model. The mean macro F1 scores for the sentiment analysis model, emotion analysis mode, and hate speech model are 72.0, 48.0, and 57.7, respectively.

Pysentimiento [13] is the Python library that supports processing with these models. To use pentimiento, analyzers are created using the “create_analyzer” function. The tweet is simply passed through the analyzer to extract the feature scores. Implicitly, the function performs pre-processing and normalizing functions. Eleven distinctive features were extracted for each tweet

entry, including positive sentiment, negative sentiment, neutral sentiment, hateful speech, targeted speech, aggressive speech, anger, fear, disgust, sadness, and surprise.

```
def get_scores(df):  
    ...  
  
    This function takes in a dataframe and adds columns for sentiment and emotion  
  
    Parameters:  
    | df: pandas dataframe  
  
    Returns:  
    | dataframe with new columns for sentiment and emotion scores  
    ...  
  
    # create new columns for sentiment and emotion scores  
    df['sentiment_neu'] = 0  
    df['sentiment_pos'] = 0  
    df['sentiment_neg'] = 0  
    df['hs_hateful'] = 0  
    df['hs_targeted'] = 0  
    df['hs_aggressive'] = 0  
    df['emo_anger'] = 0  
    df['emo_fear'] = 0  
    df['emo_disgust'] = 0  
    df['emo_sadness'] = 0  
    df['emo_surprise'] = 0  
  
    # iterate through the dataframe and get the sentiment and emotion scores  
    for index, row in df.iterrows():  
        text = row['text']  
        sentiment = get_sentiment(text)  
        emotion = get_emotion(text)  
        hate_speech = get_hate_speech(text)  
        df.at[index, 'sentiment_neu'] = sentiment['NEU']  
        df.at[index, 'sentiment_pos'] = sentiment['POS']  
        df.at[index, 'sentiment_neg'] = sentiment['NEG']  
        df.at[index, 'hs_hateful'] = hate_speech['hateful']  
        df.at[index, 'hs_targeted'] = hate_speech['targeted']  
        df.at[index, 'hs_aggressive'] = hate_speech['aggressive']  
        df.at[index, 'emo_anger'] = emotion['anger'] if 'anger' in emotion else 0  
        df.at[index, 'emo_fear'] = emotion['fear'] if 'fear' in emotion else 0  
        df.at[index, 'emo_disgust'] = emotion['disgust'] if 'disgust' in emotion else 0  
        df.at[index, 'emo_sadness'] = emotion['sadness'] if 'sadness' in emotion else 0  
        df.at[index, 'emo_surprise'] = emotion['surprise'] if 'surprise' in emotion else 0  
  
    return df
```

Figure 4.5. Adding features to tweets for prediction

4.1.4. Model Training

As mentioned before, the goal of the models is to predict the event type and disorder type for a given tweet. A fully connected neural network that takes the combined embedding of the tweet text and features was used. All training took place in Google Colab with a TPU to enhance the training process. To produce two models the following steps were followed:

Create numerical representations of data

DistilBERT is a lightweight version of the transformer model BERT. DistilBERT is trained to create embedding from textual data that are representative of the complex dependencies within the data [14]. The text data was tokenized with the DistilBERT AutoTokenizer to get input identification numbers. The classes for disorder type and event type were encoded in their respective training processes to create the labels.

```
# transform data to numerical representations|
inputs, labels = dataset_to_embeddings("train.csv")
text_input_ids = inputs[0]
numerical_features = inputs[1]
attention_masks = inputs[2]
```

Figure 4.6. Creating numerical representation for embedding

After tokenizing, the pre-trained model was used to create textual embedding with an embedding dimension of 768. The numerical features were passed through a simple neural network model that takes the features and changes them to embeddings.

```
def create_numerical_embeddings(numerical_feature_dim, hidden_dim, numerical_features):
    fc_numerical = nn.Sequential(
        nn.Linear(numerical_feature_dim, hidden_dim),
        nn.ReLU(),
        nn.Dropout(0.3)
    )

    embeddings = fc_numerical(numerical_features)

    return embeddings
```

Figure 4.7. Model for changing features to embedding

```
# Get text embeddings from BERT
text_embeddings = bertweet(input_ids=text_input_ids, attention_mask=attention_masks).last_hidden_state
```

Figure 4.8. Embedding for texts

The numerical and textual embedding are concatenated to form a final combined embedding that will be used by the model.

```
combined_embeddings = torch.cat((text_embeddings, numerical_embeddings), dim=1)
```

Figure 4.9. Combining embedding for training

Building a fully connected neural network model

A fully connected neural network that is inherited from the `nn.Module` neural network base class was created with a forward method that takes the combined embedding of the text and the features as shown below.

```

class CombinedModel(nn.Module):
    def __init__(self, hidden_dim, output_dim, input_dim, num_hidden_layers):
        """
        A fully connected model that combines text embeddings from BERTweet with numerical features to predict the disorder type.

        Args:
            input_dim (int): Dimension of the input features.
            hidden_dim (int): Dimension of the hidden layers.
            num_hidden_layers (int): Number of hidden layers.
            output_dim (int): Dimension of the output layer.
        """
        super(CombinedModel, self).__init__()

        # Fully connected layers for combined features
        self.fc_combined = nn.Sequential()
        self.fc_combined.append(nn.Linear(input_dim, hidden_dim))
        self.fc_combined.append(nn.ReLU())
        self.fc_combined.append(nn.Dropout(0.3))
        for _ in range(num_hidden_layers - 1): # Subtract 1 to account for the first layer
            self.fc_combined.append(nn.Linear(hidden_dim, hidden_dim))
            self.fc_combined.append(nn.ReLU())
            self.fc_combined.append(nn.Dropout(0.3))
        self.fc_combined.append(nn.Linear(hidden_dim, output_dim)) # Output layer

    def forward(self, combined_embeddings):
        """
        Forward pass of the model.

        # Process combined embeddings
        output = self.fc_combined(combined_embeddings)

        return output

```

Figure 4.10. Forward pass for combined embedding

Afterward, the training loop that implements the backward pass and optimizer for the data was created.

```

class Estimator(BaseEstimator, TransformerMixin):
    def __init__(self, input_dim=896,
                  hidden_dim=128, output_dim=4, num_epochs=10, lr=0.001, batch_size=32, num_hidden_layers=2):
        # consider adding number of hidden layers
        self.input_dim = input_dim
        self.hidden_dim = hidden_dim
        self.output_dim = output_dim
        self.num_epochs = num_epochs
        self.lr = lr
        self.batch_size = batch_size
        self.num_hidden_layers = num_hidden_layers
        self.model = CombinedModel(self.hidden_dim, self.output_dim, self.input_dim, self.num_hidden_layers)
        self.criterion = nn.CrossEntropyLoss()
        self.optimizer = torch.optim.Adam(self.model.parameters(), lr=self.lr)

    def fit(self, X, y):
        X = torch.tensor(X)
        y = torch.tensor(y)
        dataset = TensorDataset(X, y)
        dataloader = DataLoader(dataset, batch_size=self.batch_size, shuffle=True)
        for epoch in range(self.num_epochs):
            self.model.train()
            running_loss = 0.0

            for i, (inputs, labels) in enumerate(dataloader):
                self.optimizer.zero_grad()
                outputs = self.model(inputs)
                loss = self.criterion(outputs, labels)
                loss.backward()
                self.optimizer.step()

                running_loss += loss.item()

            epoch_loss = running_loss / len(dataloader)
            print(f'Epoch [{epoch+1}/{self.num_epochs}], Loss: {epoch_loss:.4f}')

        return self

    def predict(self, X):
        X = torch.tensor(X)
        self.model.eval()
        with torch.no_grad():
            outputs = self.model(X)
            probabilities = F.softmax(outputs, dim=1) # Get probabilities
            _, predicted = torch.max(outputs.data, 1)
        return predicted.cpu().numpy()

    def predict_proba(self, X):
        X = torch.tensor(X)
        self.model.eval()
        with torch.no_grad():
            outputs = self.model(X) # Assuming attention_mask is None
            probabilities = F.softmax(outputs, dim=1)
        return probabilities.numpy()

```

Figure 4.11. Training loop and other methods for model

Finding the best parameters using GridSearchCV

GridSearchCV was used to ensure that the best parameters were used to train the model. GridSearchCV iteratively trains a model with different sets of parameters to ensure that the right set amongst the given parameters is chosen. To train with GridSearchCV a stratified subset of the training data that includes all classes was used.

```
param_grid = {
    'hidden_dim': [64, 128],
    'num_hidden_layers': [1, 2, 3],
    'lr': [1e-4, 1e-3],
    'num_epochs': [5, 10],
    'batch_size': [16, 32]
}

param_grid_two = {
    'hidden_dim': [64, 128, 256],
    'num_hidden_layers': [1, 2, 3, 4],
    'lr': [1e-2, 1e-3, 1e-4, 1e-5],
    'num_epochs': [5, 10, 12],
    'batch_size': [16, 32, 64]
}

# Define the scoring dictionary
scoring = {
    'accuracy': 'accuracy',
    'precision_macro': make_scorer(precision_score, average='macro', zero_division = 0),
    'recall_macro': make_scorer(recall_score, average='macro', zero_division = 0),
    'f1_macro': make_scorer(f1_score, average='macro', zero_division = 0),
    'precision_weighted': make_scorer(precision_score, average='weighted', zero_division = 0),
    'recall_weighted': make_scorer(recall_score, average='weighted', zero_division = 0),
    'f1_weighted': make_scorer(f1_score, average='weighted', zero_division = 0),
}
```

Figure 4.12. Defining parameters for GridSearchCV

```
model = Estimator()
grid_search = GridSearchCV(model, param_grid_two, cv=5, scoring=scoring, refit='f1_weighted')
```

```
grid_search.fit(combined_embeddings, labels)
```

Figure 4.13. Training with GridSearchCV

For the disorder type model, the best parameters given were ‘{'batch_size': 16, 'hidden_dim': 128, 'lr': 0.001, 'num_epochs': 12, 'num_hidden_layers': 2}'. For the event type model, the best parameters given were ‘{'batch_size': 16, 'hidden_dim': 128, 'lr': 0.001, 'num_epochs': 10, 'num_hidden_layers': 1}'.

Training Model using K-Fold Cross Validation

Given the best set of parameters for each model, the entire training dataset was trained using K-Fold Cross Validation. Using this method ensures that the data is used well for training, especially given that the data is skewed and includes underrepresented classes. Using K-Fold also provides a good method of evaluation. In this implementation, k=5.

```
disorder_model = Estimator(input_dim=896, hidden_dim=128, output_dim=4, num_epochs=12, lr=0.001, batch_size=16, num_hidden_layers=2)
```

```
event_model = Estimator(input_dim=896, hidden_dim=128, output_dim=4, num_epochs=10, lr=0.001, batch_size=16, num_hidden_layers=1)
```

Figure 4.14. Model definitions for disorder type and event type

```

for train_index, val_index in kf.split(combined_embeddings):
    X_train, X_val = combined_embeddings[train_index], combined_embeddings[val_index]
    y_train, y_val = labels[train_index], labels[val_index]

    model.fit(X_train, y_train)
    y_pred = model.predict(X_val)

    accuracy = accuracy_score(y_val, y_pred)
    precision_macro = precision_score(y_val, y_pred, average='macro')
    recall_macro = recall_score(y_val, y_pred, average='macro')
    f1_macro = f1_score(y_val, y_pred, average='macro')
    precision_weighted = precision_score(y_val, y_pred, average='weighted')
    recall_weighted = recall_score(y_val, y_pred, average='weighted')
    f1_weighted = f1_score(y_val, y_pred, average='weighted')

    accuracies.append(accuracy)
    precisions_macro.append(precision_macro)
    recalls_macro.append(recall_macro)
    f1s_macro.append(f1_macro)
    precisions_weighted.append(precision_weighted)
    recalls_weighted.append(recall_weighted)
    f1s_weighted.append(f1_weighted)

```

Figure 4.15. K-fold Cross Validation Loop

Evaluation Results

The following table shows the results of the evaluation for the disorder type model.

Evaluation	Accuracy	Recall	Precision	Weighted F1
Score	0.7337	0.7337	0.7390	0.7219

Table 4.1. Evaluation results for disorder-type model

The following table shows the results of the evaluation for the event type model.

Evaluation	Accuracy	Recall	Precision	Weighted F1
Score	0.6662	0.6662	0.6695	0.6440

Table 4.2. Evaluation results for event-type model

The resulting models are saved to be used in the conflict monitoring application.

4.2. Building the Web Application

The web application consists of a simple web dashboard and a containerized flask application in the backend that runs the model. The front-end frameworks used were React.js, d3.js, and Tailwind CSS. These front-end frameworks were chosen due to their flexibility and ease of use. The backend framework used is flask for the APIs.

4.2.1. Backend implementation

Due to the restrictions on live streaming data from the X API, tweets from two countries were collected as test cases for the model and use cases for the end user. The data collection component follows the same process for collecting data using the X API. The collected data is stored as files. When a user initiates monitoring the following steps occur in the backend:



Figure 4.16. Backend Application process

Filtering by unrest score

Each tweet is passed through the civil unrest score model [15]. All tweets with a score less than 0.5 are discarded.

```

def filter_by_unrest_score(text):
    tokens = normalizeTweet(text)
    tokens = tokenizer(tokens, padding=True, truncation=True, return_tensors="pt")
    input_ids, attention_mask = tokens["input_ids"].clone().detach(), tokens["attention_mask"]

    # Get text embeddings from BERTweet
    text_embeddings = bertweet(input_ids=input_ids, attention_mask=attention_mask).last_hidden_state

    # Pool the embeddings (use the [CLS] token representation or mean pooling)
    text_embeddings = text_embeddings.mean(dim=1)

    # Make predictions
    text_embeddings = text_embeddings.to(device)

    with torch.no_grad():
        predictions_proba = civil_unrest_model.predict_proba(text_embeddings)

    discuss_unrest = predictions_proba > 0.5

    unrest_tweets = [text for i in range(len(discuss_unrest)) if discuss_unrest[i]]

    return unrest_tweets

```

Figure 4.17. Filtering tweets by unrest score

Keyword Retrieval

KeyBERT is a transformer-based model used to extract keywords from text. This model is accessed through the keybert python library. Apart from extracting keywords with the transformer model, the system implements extracting hashtags as keywords using simple regular expressions.

```

def save_keywords(tweets):
    if tweets:
        keywords = kw_model.extract_keywords(tweets)
        for keyword in keywords:
            if keyword[0] in keywords_dict:
                keywords_dict[keyword[0]] += 1
            else:
                keywords_dict[keyword[0]] = 1

        pattern = re.compile(r"^#(\w*[a-zA-Z0-9])+$.")

        hashtags = [word for word in tweets.split() if pattern.match(word)]
        for hashtag in hashtags:
            if hashtag in keywords_dict:
                keywords_dict[hashtag] += 1
            else:
                keywords_dict[hashtag] = 1

```

Figure 4.18. Extracting hashtags

Disorder and event type prediction

This module loads the disorder type and event type model. It also performs data tokenization and embedding as shown in section 4.1.3.

```

def predict_scores(embeddings):
    predictions_disorder = disorder_model.predict(embeddings)
    confidence_disorder = disorder_model.predict_proba(embeddings).max(axis=1)
    predictions_event = event_model.predict(embeddings)
    confidence_event = event_model.predict_proba(embeddings).max(axis=1)

    predictions = {
        "disorder": [predictions_disorder.item(), confidence_disorder.item()],
        "event": [predictions_event.item(), confidence_event.item()]
    }

    return predictions

```

Figure 4.19. Predicting using model

Data Aggregation

After each tweet predicts its disorder type and event type, the cumulative results are what is shown to the user. This cumulating is done using simple majority voting among the predictions. This means the result given to the user is the result that most of the tweets predict. This is done after each tweet is predicted. The final prediction is added to the keywords to be displayed to the user.

```
def majority_voting_multiclass(df_combined):
    disorder_confidence = df_combined["confidence_disorder"]
    event_confidence = df_combined["confidence_event"]
    disorder = df_combined["predictions_disorder"]
    event = df_combined["predictions_event"]

    # Perform majority voting for disorder predictions
    disorder_counter = Counter(disorder)
    final_prediction_disorder = disorder_counter.most_common(1)[0][0]

    # Perform majority voting for event predictions
    event_counter = Counter(event)
    final_prediction_event = event_counter.most_common(1)[0][0]

    # Determine unique classes
    disorder_classes = np.unique(disorder)
    event_classes = np.unique(event)

    # Initialize dictionaries to store weighted sums and counts
    weighted_sums_disorder = {clas: 0 for clas in disorder_classes}
    weighted_counts_disorder = {clas: 0 for clas in disorder_classes}

    weighted_sums_event = {cls: 0 for cls in event_classes}
    weighted_counts_event = {cls: 0 for cls in event_classes}

    # Accumulate weighted sums and counts for disorder
    for confidence, prediction in zip(disorder_confidence, disorder):
        weighted_sums_disorder[prediction] += confidence
        weighted_counts_disorder[prediction] += 1

    # Accumulate weighted sums and counts for event
    for confidence, prediction in zip(event_confidence, event):
        weighted_sums_event[prediction] += confidence
        weighted_counts_event[prediction] += 1

    overall_confidence_disorder = weighted_sums_disorder[final_prediction_disorder] / weighted_counts_disorder[final_prediction_disorder]
    overall_confidence_event = weighted_sums_event[final_prediction_event] / weighted_counts_event[final_prediction_event]

    keywords = sorted(keywords_dict.items(), key=lambda x: x[1], reverse=True)

    # create list of dictionaries with the to 20 keywords as 'text' and their frequencyx10 as 'value'
    top_keywords = [{"text": keyword[0], "value": keyword[1]} for keyword in keywords[:20]]

    data_send = {
        "final_prediction_disorder": disorders[final_prediction_disorder],
        "final_prediction_event": events[final_prediction_event],
        "confidence_disorder": overall_confidence_disorder,
        "confidence_event": overall_confidence_event,
        "top_keywords": top_keywords
    }
```

Figure 4.20. Aggregating data using majority vote

Communicating with frontend

As per the functional requirements, the ability of the system to communicate data to the user in real time is important. To do this the system uses web sockets to communicate with the front-end client. This is done using the flask socket io library.

```
socketio = SocketIO(app, cors_allowed_origins="*")
```

Figure 4.21. Initializing socket for communication

After every prediction, the results are sent to the front-end application to be displayed to the user.

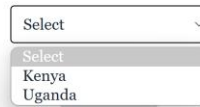
```
# send the final predictions to the front end  
with app.app_context():  
    socketio.emit("predict", {'data': data_send})
```

Figure 4.22. Using socket for communication

4.2.2. Frontend Application

Once a user is authenticated, the user can choose the country whose conflict they are interested in monitoring.

Choose the country to monitor



Select

Select

Kenya

Uganda

Figure 4.23. Select country page

After selecting a country, a user can view the predicted disorder and event types along with their confidence scores. The user can also see the keywords associated with the tweets. Their size and color indicate their frequency in the processed tweets.

Conflict Monitoring System Dashboard

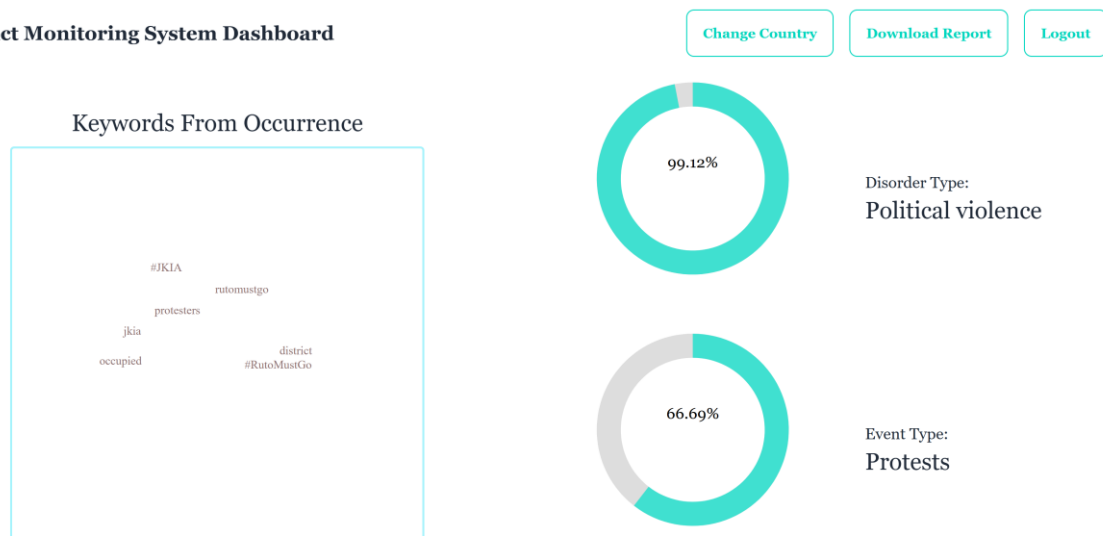


Figure 4. 24. View dashboard page

The user can choose to download the report for further analysis or for keeping.

Conflict Monitoring System Dashboard

Keywords From Occurrence

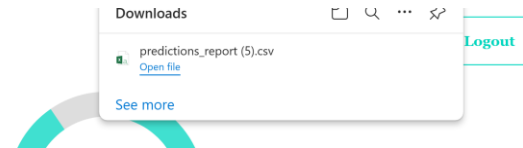


Figure 4.25. User download of report file

The live communication between the back and frontend was implemented using npm socket io library. The live communication opens when a user starts monitoring an area.

```
const socket = io("https://capstone-2t76ybystq-uc.a.run.app");
```

Figure 4.26. Socket communication on frontend application

The arc tween and cloud get updated based on the information received by the socket.

```
socket.on("predict", (data) => {  
  data = data.data;  
  if (!isNaN(data.confidence_disorder) && data.final_prediction_disorder !== undefined ||  
      !isNaN(data.confidence_event) && data.final_prediction_event !== undefined) {  
    setPredictions([data]);  
    setHistory(prevHistory => [...prevHistory, data]);  
  }  
});
```

Figure 4. 27. Updating predictions, confidence scores, and keywords when data is received from socket

```
function updateArc(newAngle) {
  foreground.transition()
    .duration(750)
    .attrTween("d", arcTween(newAngle));
  confidenceText.text(`${(confidence * 100).toFixed(2)}%`);
}
```

Figure 4.28. Update arc when data is updated

```
useEffect(() => {
  const svgElement = d3.select(ref.current)
    .attr("viewBox", `0 0 700 500`)
    .attr("preserveAspectRatio", "xMidYMid meet")
    .attr("class", "word-cloud");

  if (keywords.length > 0) {
    const wordsData = processWords(keywords);
    drawWordCloud(svgElement, wordsData);
  }
}, [keywords]);
```

Figure 4.29. Update word cloud when keyword is updated

CHAPTER 5: TESTING AND RESULTS

The method of testing that would be used to verify the set of functional requirements for the frontend and backend system is manual testing, specifically, white box testing. This manual testing method ensures that the internal parts of the system are optimized and working as required.

Two use cases will be used to test the machine learning model, tweets from Kenya between 11:30 am and 12:00 pm on 23rd July 2024, and tweets from Uganda between 8:58 am and 9:00 am on the 23rd of July 2024.

5.1. Component Testing

This testing was done to ensure that the components interact well. One component that was tested was the socket component. To test this component, the frontend application sends a message to the backend application and the backend application in turn sends a response to the frontend application.

```
socket.on("connect", () => {  
  console.log("Connected to server");  
});
```

Figure 5.1. Frontend message to server

```
@socketio.on("connect")
def connected():
    """event listener when client connects to the server"""
    print(request.sid)
    print("connected")
    emit("connect",{"data":f"id: {request.sid} is connected"})
```

Figure 5.2. Backend application response to frontend application

Figure 5.3. shows the frontend component has connected and *Figure 5.4.* shows the backend component has been connected.

Connected to server

[Dashboard.js:64](#)

Figure 5.3. The frontend component connected to the backend server

```
INFO:werkzeug:127.0.0.1 - - [31/Jul/2024 19:58:09] "GET /socket.io/?EIO=4&transport=polling&t=P4ANnQc HTTP/1.1" 200 -
H9i1aKG_PiYPPJzoAAAG
connected
```

Figure 5.4. The backend component connected to the frontend component

Another component test was to test the endpoint that sends the predictions.

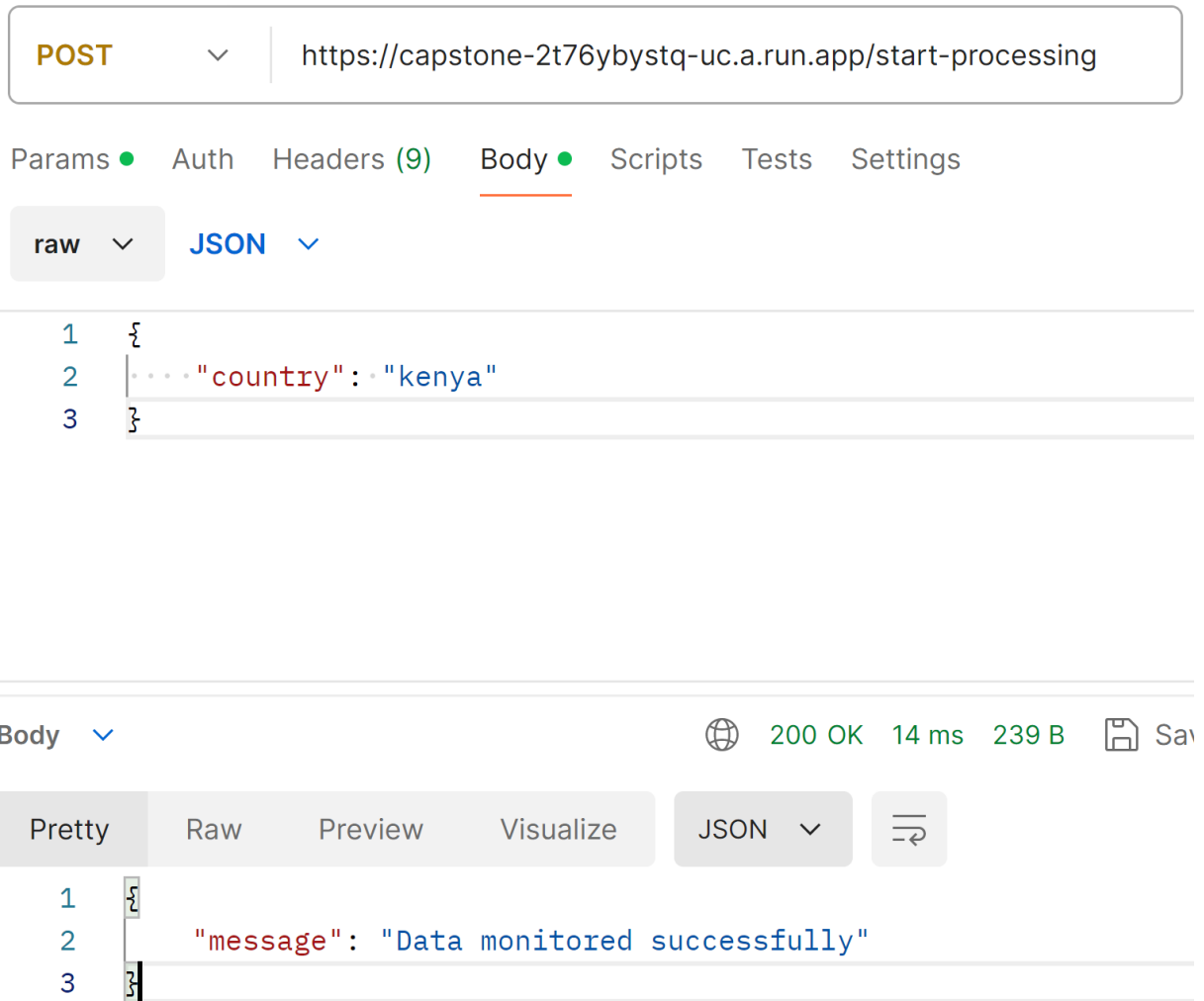


Figure 5.5. Backend endpoint component testing

The component testing also included authentication tests for user login and register. These are shown in appendices H and I.

5.2. System Testing

To conduct a full system testing, the user will follow a series of steps. These steps are meant to test the full functionality of the systems. The steps are defined in Figure 5.6. as follows:

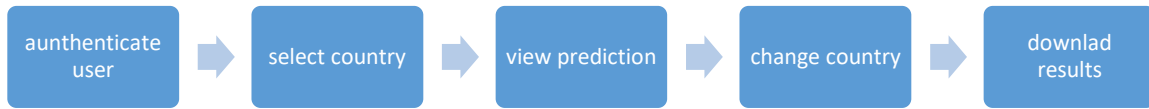


Figure 5. 6. System Testing steps

The results of these steps are shown in appendices C to G.

5.3. User Testing

To perform user testing, the system was presented to two professionals in the conflict field. The users indicated that the system was intuitive to use. They particularly indicated that the word cloud was useful in understanding the results of the predictions. The users were concerned with the availability of just two countries, as this limits the dashboard's ability.

5.4. Model Testing

To test the model, the results of the tweets are compared against ACLED results. Appendix E shows that the results from Kenya indicate Political Violence as the disorder type and Protests as the event type. The results from ACLED show that the disorder type is Political Violence, and the event type is Violence against civilians. The mismatch in event type is expected as the confidence score for the event type is between 0.6 and 0.7.

event_id	event_date	year	time_precision	disorder_type	event_type
KEN15085	23-Jul-24	2024	1	Political violence	Violence against civilians

Figure 5.7. Results for Kenya from ACLED API

Appendix F shows that the results from Uganda indicate Demonstrations as the disorder type and Protests as the event type. The results from ACLED show that the disorder type is Demonstrations, and the event type is Demonstrations. The confidence score for disorder type and event type are 0.9653 and 0.8303. This explains the correct match of the events.

CHAPTER 6: CONCLUSIONS AND RECOMMENDATIONS

6.1. Summary

Conflict monitoring systems are essential in analyzing conflict and preparing for various responses. While traditional monitoring systems are useful, they are biased since they may be opinion-based instead of fact-based. Sentiment analysis is a useful tool for understanding texts. This project used sentiment analysis to understand the tweets produced by people in areas where conflict may be ensuing. The system also uses neural networks and transformer models to find the disorder type and event type as provided by ACLED.

The complete system allows a user to choose to access a web-based user dashboard to view the results of the models' predictions. The users can specify the geographical regions of interest they would like to observe. The users can also download reports containing the aggregate probabilities over time for later viewing and analysis. Apart from these services available to clients the system can process tweets using machine learning models that have been deployed to the cloud as a service. This has been made scalable by employing Docker containerization and running in a cloud environment.

The disorder type and event type prediction models were built by using sentiment and BERT text embedding as features for neural network. The models had F1 scores of 0.7219 and 0.6440, respectively. The use cases for testing showed promising results for the system.

The system lacks some stated functional requirements. It does not allow users to manage predictions based on time and specific regions instead of whole count

6.2. Limitations

While the system is useful, it lacks some features that would enhance its use. One of such features is real-time monitoring. Currently, the system can only work for two use cases. This is due to the limitation of the X API that restricts the live streaming of tweets for accounts with basic API access. Another one is that the system does not show a use case where no type of conflict or unrest is happening. This limitation is also due to a feature of the X API where tweets cannot be searched for a particular country meaning data cannot be collected for a specific country with no type of unrest. Another limitation is the limited event types the system can detect. Also, ACLED has a sub-event type that was not included in the system. These limitations are due to lack of data that is representative of every event and sub-event type.

6.3. Future Works

An important inclusion to the conflict monitoring system would be ensuring real-time monitoring as this would be particularly useful for real-life applications. This can be implemented if a higher version of the X API is used.

Another useful feature that can be included in the system is going beyond english-speaking countries to include all countries in the Sub-Saharan Africa region. This will form a comprehensive system.

The system will benefit from the inclusion of a sub-event type prediction model as the sub-event types are more specific and contain situations such as airstrikes and sexual violence against women.

For the dashboard, improvements to the user management system can be made to allow users to save reports in the application instead of downloading them. Future work will

consider integrating functionalities that improve user experience and value from using the data.

References

- [1] *2023 Social Media Report*. Statista, 2023
- [2] ACLED. "Armed Conflict Location & Event Data Project (ACLED) Codebook." *ACLED Home Knowledge Base*. ACLED, 2 June 2023
- [3] Affa'a-Mindzie, Mireille. "Preventing conflicts in Africa: early warning and response." *meeting note*. New York, NY: International Peace Institute. 2012.
- [4] Akter, Shahriar, and Samuel Fosso Wamba. "Big data and disaster management: a systematic review and agenda for future research." *Annals of Operations Research* 283 (2019): 939-959.
- [5] Chinta, Abhinav, et al. ["Study of Manifestation of Civil Unrest on Twitter."](#)¹ *Proceedings of the 2021 EMNLP Workshop W-NUT: The Seventh Workshop on Noisy User-generated Text*, Association for Computational Linguistics, 11 Nov. 2021, pp. [396-409](#)².
- [6] Duan, Jiexiong, Weixin Zhai, and Chengqi Cheng. "Crowd detection in mass gatherings based on social media data: A case study of the 2014 shanghai new year's eve stampede." *International journal of environmental research and public health* 17.22 (2020): 8640.
- [7] Frangonikolopoulos, C. A., & Chapsos, I. (2012). Explaining the role and the impact of the social media in the Arab Spring. *Global Media Journal: Mediterranean Edition*, 7(2).
- [8] Gates, Scott, et al. "Consequences of civil conflict." (2010).
- [9] Green, Kesten C., and J. Scott Armstrong. "The ombudsman: Value of expertise for forecasting decisions in conflicts." *Interfaces* 37.3 (2007): 287-299.

- [10] IBM. "Architectural Characteristics of Web-Based Applications." IBM Documentation, www.ibm.com/docs/en/db2-for-zos/12?topic=environment-architectural-characteristics-web-based-applications. Accessed 31 July 2024.
- [11] Kavu, T. D., Nyamandi, T. G., Chirinda, A., Rugube, T. T., & Zishumba, K. (2017). Human Action Prediction Using Sentiment Analysis on Social Networks. *International Journal of ICT Research in Africa and the Middle East (IJICTRAME)*, 6(2), 14-28.
- [12] Kilcullen, D., & Courtney, A. (2014). *Big Data, Small Wars, Local Insight: Designing for Development with Conflict-Affected Communities*. McKinsey on Society.
- [13] Pérez, Juan Manuel, et al. "pysentimiento: A Python Toolkit for Opinion Mining and Social NLP tasks." *arXiv*, 26 Oct. 2023, doi:10.48550/arXiv.2106.09462.2
- [14] Sanh, Victor, et al. "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter." *arXiv preprint arXiv:1910.01108* (2019).
- [15] Sech, Justin, et al. "Civil unrest on Twitter (CUT): A dataset of tweets to support research on civil unrest." *Proceedings of the Sixth Workshop on Noisy User-generated Text (W-NUT 2020)*. 2020.
- [16] Sub-Saharan Africa | The Armed Conflict Survey 2022. (n.d.). IISS. <https://www.iiss.org/online-analysis/online-analysis//2022/11/acs-2022-sub-saharan-africa>
- [17] Tetlock, Philip E. "Expert political judgment: How good is it? How can we know?-New edition." (2017): 1-368.

Appendix

Appendix A: Semi-Structured Interview Guide

INSIGHTS INTO CONFLICT MONITORING IN SUB-SAHARAN

Thank you for agreeing to participate in this interview. We are interviewing you to understand better what type of information is more relevant to [organization name] when assessing conflict-afflicted areas and how this information can be presented to aid [organization name] in decision-making. During this interview, answers will not be perceived as right or wrong since we are interested in your opinions.

Do you have any questions about the interview?

May I start the recording?

This guide only represents the main themes to be discussed with the participants and, as such, does not include the various prompts that may also be used. Non-leading and general prompts will also be used, such as "Can you please tell me a little bit more about that?" and "What does that look like for you."

Establishing Rapport

Countries like Sudan and the Democratic Republic of Congo are currently facing conflict. What do you think governments and African organizations like the African Union can implement to manage or prevent the outbreak of conflicts in the Sub-Saharan African region?

1. How frequently does your organization deal with problems involving conflict or conflict-afflicted zones in Sub-Saharan Africa?
2. What roles do your organization play in conflict-afflicted areas?
3. Are conflict monitoring systems useful for managing conflicts?
 - a. Why do you think so?
4. To the best of your knowledge, what is the current state of conflict monitoring technology in Sub-Saharan Africa?
5. Are conflict monitoring systems a priority in organizations like yours?

6. How intense does the conflict have to be for it to be recognized by or acted upon by your organization?
7. During conflicts, the civilians in the conflict-affected areas are usually deprived of their needs, like food and clean water.
 - a. Is knowing the humanitarian needs of the people affected by conflict relevant to decision-makers? Why so?
 - b. What do you think about a system that monitors the humanitarian needs of the people affected by the conflict?
8. During conflicts, civilians in the conflict-affected areas may choose to migrate within their country or, in extreme situations, migrate to other, more peaceful countries.
 - a. Is knowing the displacement statistics of the people relevant to decision-makers? How so?
 - b. What kind of information about displacement of people in the conflict-affected is most relevant?
9. During intense conflict, there are often casualties that may occur due to changes in the intensity of violence.
 - a. As decision-makers, is it useful to have information about the statistics of casualties and the changes in the intensity of violence towards civilians?
 - b. Is it useful to know this information about the groups involved in the conflict?
10. Are conflict actor profiles useful for conflict monitoring?
 - a. What information about the conflict actor(s) is most useful for decision-makers?
11. Based on your organization's needs, what is the most vital information or data you need to resolve conflict problems?

12. How readily available is this data?
13. Do you think social media is a credible source of data for conflict monitoring?
14. Does your organization have the technology to get this data? Can you tell me more about this technology?
15. Would your organization be interested in working with a student to build a machine learning tool that supports conflict monitoring in Sub-Saharan Africa?
 - a. What type of collaboration would be most suitable?

Appendix B: Register and Login Implementation

```
@app.route("/register_user", methods=['POST'])
def register_user():
    user_id = request.json['user_id']
    user_ref = users.document(user_id)
    user = user_ref.get()

    # check if student exists
    if user.exists:
        return jsonify({"message": "exists"}), 400

    # add user to database
    users.document(user_id).set(request.json)
    return jsonify({"message": "created"}), 201

''' ...

@app.route("/validate_user", methods=['POST'])
def validate_user():
    user_id = request.json['user_id']
    user_ref = users.document(user_id)
    user = user_ref.get()

    # check if student exists
    if not user.exists:
        return jsonify({"message": "not exist"}), 400
    if user.to_dict()['password'] != request.json['password']:
        return jsonify({"message": "wrong password"}), 400

    return jsonify({"message": "exists"}), 200
```


Appendix C: Register and Login for User

Register

Register

Passwords do not match.

[Login Instead](#)

Appendix D: Select Country

Choose the country to monitor



Start

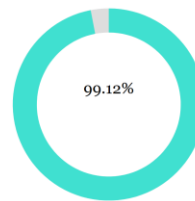
Appendix E: Real time monitoring of Kenya Tweets

Conflict Monitoring System Dashboard

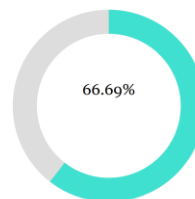
Change Country

[Download Report](#)[Logout](#)

Keywords From Occurrence



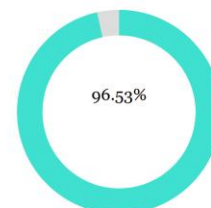
Disorder Type:
Political violence



Event Type:
Protests

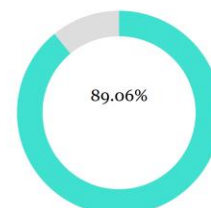
Appendix F: Real time monitoring of Uganda Tweets

Keywords From Occurrence



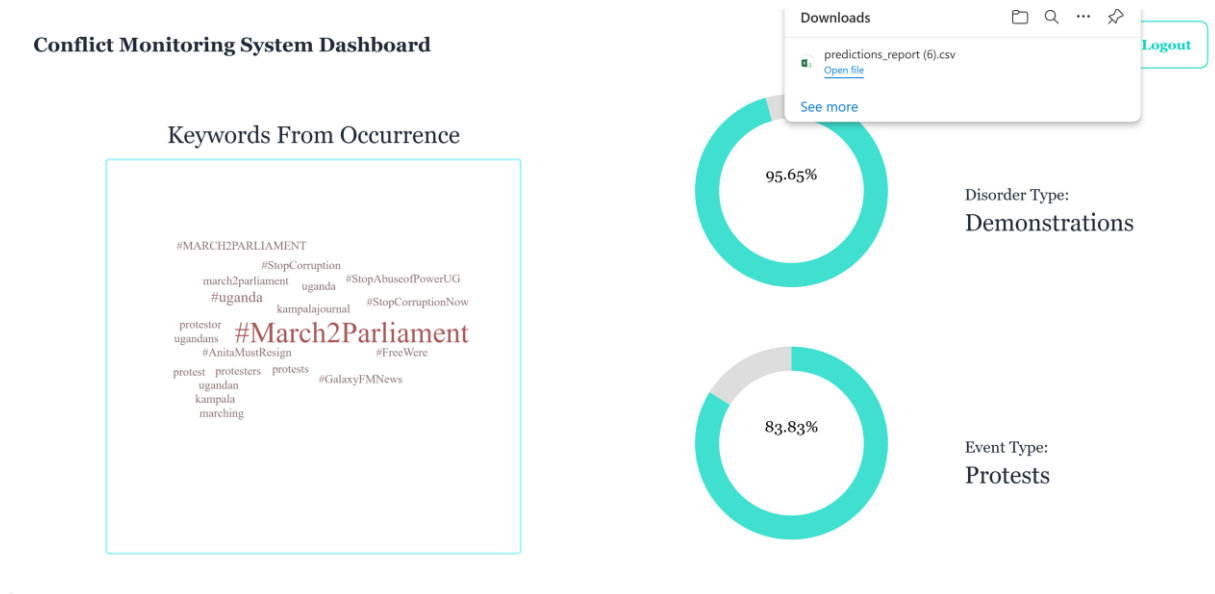
Disorder Type:

Demonstrations



Event Type:
Protests

Appendix G: Downloading report file



Appendix H: Login component testing

Login

Login

Username or password is incorrect.

[Register Instead](#)

Appendix I: Register component testing

Register

Password must be at least 8 characters long and include at least one number, one uppercase and one lowercase letter.

[Login Instead](#)